

JAVA AI ENGINEER · MÓDULO 3

Taller evaluativo

Backend profesional con Spring Boot

Proyecto **VetTurno**

La agenda digital de Veterinaria Huellitas

Entrega	Tecnologías	Producto verificable
Repositorio GitHub	Java 17 · Spring Boot · Maven · MySQL · JPA · JWT · Swagger	API de citas documentada, validada, segura y persistente

Tu reto

Doña Marta confía en ti para ordenar el trabajo cotidiano de su veterinaria. No vas a repetir el gestor de productos ni el proyecto universitario: vas a transformar una necesidad cercana en una API que otra persona pueda comprender, probar y defender.

Antes de comenzar

Qué se evalúa

Este taller reúne lo trabajado en las clases 9 a 12: proyecto Spring Boot con Maven, arquitectura por capas, API REST con GET y POST, JPA/Hibernate y MySQL, relaciones, repositorios, DTO, seguridad JWT con roles, validaciones, manejo global de errores, Swagger/OpenAPI, README y publicación en GitHub.

Cómo trabajar este documento

- Lee primero la historia, el alcance y el contrato funcional. Empieza a programar cuando comprendas qué necesita la veterinaria.
- Avanza por las ocho partes en orden. Cada punto de control prepara el trabajo de la parte siguiente.
- Usa los nombres y reglas de VetTurno. La meta es transferir lo aprendido a un dominio nuevo, no renombrar el proyecto de clase.
- Prueba después de cada parte y conserva evidencia. Que la aplicación inicie no demuestra por sí solo que el flujo funciona.
- Puedes usar IA para comprender, revisar o diagnosticar. Toda decisión incorporada debe poder explicarse con tus palabras.

Correspondencia con las clases

Clase	Aprendizajes que se evidencian
9 · Spring Boot y REST	Proyecto Maven, servidor embebido, capas, inyección por constructor y endpoints GET/POST.
10 · Persistencia	JPA/Hibernate, MySQL, entidades, relaciones, JpaRepository, consultas derivadas y DTO.
11 · Seguridad	Registro, login, BCrypt, JWT, filtro, modo stateless, roles USER/ADMIN y rutas protegidas.
12 · Calidad y entrega	Swagger/OpenAPI, DTO de entrada, Bean Validation, errores globales, estados HTTP, README y GitHub.

Qué significa publicar en este taller

Publicar significa dejar VetTurno disponible en un repositorio GitHub y documentar en el README cómo ejecutarlo, configurarlo y probarlo. El despliegue en la nube no es obligatorio. Docker tampoco forma parte de la evaluación: encontrarás una práctica opcional, sin puntaje, al final del documento.

La historia: Veterinaria Huellitas

Doña Marta abrió Veterinaria Huellitas hace seis años en un barrio donde conoce por su nombre a muchas familias y a sus mascotas. Ella atiende la clínica con el doctor Andrés y con Paula, quien recibe llamadas, responde mensajes y organiza las citas.

La agenda todavía vive entre un cuaderno y conversaciones de WhatsApp. Cuando el día está ocupado, Paula puede reservar dos consultas para el mismo veterinario a la misma hora, escribir mal el nombre de una mascota o perder el teléfono de su responsable. El problema no es falta de cuidado: es que la información está dispersa.

Tu misión es construir **VetTurno**: una API que permita registrar responsables, mascotas y veterinarios, agendar citas sin cruces de horario y consultar la agenda con información clara y persistente.

Personas usuarias y necesidades

Persona	Necesidad dentro de VetTurno
Paula · Recepcionista · USER	Registra responsables y mascotas, agenda citas y consulta la agenda sin administrar cuentas ni datos exclusivos.
Doña Marta · Administradora · ADMIN	Puede realizar las tareas de recepción y además registrar veterinarios.
Doctor Andrés · Veterinario	Necesita que sus citas aparezcan ordenadas y relacionadas con la mascota correcta. No inicia sesión en este MVP.
Persona evaluadora	Debe poder ejecutar la API, comprender sus reglas y verificar el flujo mediante Swagger y el README.

Alcance del MVP

Debe incluir	Queda fuera de alcance
Registro y login con roles USER y ADMIN. Responsables, mascotas, veterinarios y citas. Prevención de horarios duplicados por veterinario. Consulta de citas por veterinario. Validación, errores, Swagger, MySQL, README y GitHub.	Historia clínica, diagnósticos o fórmulas. Pagos, facturación, inventario o tienda de mascotas. Recordatorios por correo o WhatsApp. Interfaz web o aplicación móvil. Despliegue cloud y Docker como requisito.

Criterios de aceptación de Doña Marta

- ☐ Las mascotas quedan relacionadas con su responsable correcto.
- ☐ Una cita siempre referencia una mascota y un veterinario existentes.
- ☐ No se permiten citas en el pasado ni dos citas para el mismo veterinario a la misma hora.
- ☐ Una persona USER no puede registrar veterinarios.
- ☐ Las citas continúan en MySQL después de reiniciar la aplicación.
- ☐ Swagger y el README permiten probar el flujo sin leer todo el código fuente.

Contrato funcional de VetTurno

Regla de diseño

Las entidades representan lo que se guarda en MySQL; los DTO definen lo que entra y sale de la API. Las respuestas deben mostrar relaciones de forma breve —por ejemplo, nombre de mascota y nombre del veterinario— para evitar ciclos JSON y datos innecesarios.

Modelo mínimo de datos

Elemento	Campos mínimos	Reglas
Propietario	id, nombre, teléfono, email	Nombre y teléfono obligatorios; email válido; puede tener varias mascotas.
Mascota	id, nombre, especie, raza, propietario	Nombre, especie y propietario obligatorios; pertenece a un solo responsable.
Veterinario	id, nombre, especialidad	Nombre y especialidad obligatorios; solo ADMIN puede registrarlo.
Cita	id, fechaHora, motivo, mascota, veterinario	Fecha futura; motivo obligatorio; no puede coincidir con otra cita del mismo veterinario.
Usuario	id, email, password, rol	Email único; contraseña con BCrypt; rol USER o ADMIN almacenado como texto.

Endpoints obligatorios

Método	Ruta	Acceso	Resultado esperado
POST	/api/auth/register	Público	Registra USER con datos válidos y devuelve un token.
POST	/api/auth/login	Público	Verifica credenciales y devuelve un JWT vigente.
POST	/api/proprietarios	USER / ADMIN	Crea un responsable; responde 201.
GET	/api/proprietarios	USER / ADMIN	Lista responsables mediante DTO.
POST	/api/mascotas	USER / ADMIN	Crea una mascota relacionada con un propietario existente; responde 201.
GET	/api/mascotas	USER / ADMIN	Lista mascotas sin anidar todas sus citas.
POST	/api/veterinarios	ADMIN	Crea un veterinario; USER recibe 403.
GET	/api/veterinarios	USER / ADMIN	Lista profesionales disponibles.
POST	/api/citas	USER / ADMIN	Agenda una cita válida y sin cruce; responde 201.
GET	/api/citas	USER / ADMIN	Devuelve la agenda completa mediante DTO.
GET	/api/citas/veterinario/{id}	USER / ADMIN	Filtra citas por veterinario.

Contratos de entrada y salida

DTO sugerido	Dirección	Campos
RegistroRequest	Entrada	email, password
LoginRequest	Entrada	email, password
AuthResponse	Salida	token
PropietarioRequest / DTO	Entrada / salida	nombre, teléfono, email; salida también incluye id

DTO sugerido	Dirección	Campos
MascotaRequest	Entrada	nombre, especie, raza, propietarioId
MascotaDTO	Salida	id, nombre, especie, raza, propietarioId, propietarioNombre
VeterinarioRequest / DTO	Entrada / salida	nombre, especialidad; salida también incluye id
CitaRequest	Entrada	fechaHora, motivo, mascotaId, veterinarioId
CitaDTO	Salida	id, fechaHora, motivo, mascota, propietario, veterinario
ApiError	Salida de error	status, mensaje, errores por campo, timestamp

Reglas de seguridad y respuesta

- Registro y login son públicos. La información de la veterinaria requiere autenticación.
- USER y ADMIN pueden registrar responsables, mascotas y citas; registrar veterinarios exige ADMIN.
- El registro siempre asigna USER, aunque el cliente intente enviar otro rol.
- El primer ADMIN se habilita de manera controlada en la base, como se trabajó en clase, y luego inicia sesión de nuevo.
- Datos inválidos o una cita cruzada: 400. Usuario sin permiso: 403. Creación: 201. Consulta o autenticación: 200. Error interno: 500 sin detalles sensibles.

Parte 1 — Preparar el proyecto de la veterinaria

Propósito

Construir una base limpia y ejecutable para VetTurno, con las dependencias del módulo, control de versiones desde el inicio y paquetes que hagan visible la responsabilidad de cada capa.

Conceptos que se repasan

Spring Boot aporta el servidor embebido y la autoconfiguración; Maven administra dependencias y construye el proyecto. La arquitectura por capas separa entrada HTTP, lógica de negocio, persistencia, contratos de datos y seguridad para que cada archivo tenga una responsabilidad comprensible.

Instrucciones paso a paso

1. Crea el proyecto con Spring Initializr usando Maven, Java 17 y la versión de Spring Boot trabajada en clase. Usa un group y artifact coherentes con VetTurno.
2. Selecciona Web/Web MVC, Data JPA, MySQL Driver, Security y Validation. Incorpora JWT y springdoc/OpenAPI con las versiones utilizadas en el módulo.
3. Abre el proyecto, espera a que Maven resuelva las dependencias y ejecuta la clase principal antes de añadir lógica.
4. Crea los paquetes model, repository, service, controller, dto, security, config y exception dentro de un único paquete base.
5. Inicia Git, agrega un .gitignore adecuado y realiza un primer commit de la base ejecutable.
6. Crea el README con el nombre VetTurno, la historia de Veterinaria Huellitas, las tecnologías y una sección “Cómo ejecutar”.

```
.\mvnw.cmd spring-boot:run    # Windows | ./mvnw spring-boot:run    #
macOS/Linux
```

```
git init · git status · git add . · git commit -m "chore: iniciar VetTurno"
```

Responsabilidad de los archivos involucrados

Archivo o carpeta	Responsabilidad
pom.xml	Declara la identidad del proyecto, Java, dependencias y proceso de construcción Maven.
VetTurnoApplication.java	Punto de entrada que inicia y autoconfigura la aplicación.
src/main/java/...	Código organizado bajo un paquete base para que Spring encuentre los componentes.
application.properties	Configuración local de la aplicación y conexión con MySQL.
.gitignore	Evita publicar binarios, archivos del IDE y configuración sensible.
README.md	Guía para comprender, ejecutar, probar y evaluar VetTurno.

Resultado esperado

Punto de control 1

VetTurno compila, el servidor inicia, los paquetes son legibles y el primer commit contiene una base limpia, sin credenciales ni archivos generados.

Lista breve de verificación

- ☐ Java 17 y Maven están configurados correctamente.
- ☐ La clase principal se encuentra dentro del paquete base.
- ☐ Cada paquete responde a una responsabilidad real.
- ☐ El repositorio local tiene un commit inicial limpio.
- ☐ El README permite reconocer la veterinaria y el propósito del proyecto.

Evidencia que debes presentar

- Captura del inicio correcto del servidor y del árbol de paquetes.
- Enlace al commit inicial.
- Explicación breve de la diferencia entre pom.xml y la clase principal.

Parte 2 — Modelar la información de Huellitas

Propósito

Representar responsables, mascotas, veterinarios y citas como un modelo relacional coherente y persistirlo en MySQL mediante JPA/Hibernate.

Conceptos que se repasan

JPA define cómo se mapean objetos a tablas y Hibernate ejecuta ese mapeo. En una relación ManyToOne, la llave foránea queda en el lado que contiene muchos registros: muchas mascotas pueden pertenecer a un responsable y muchas citas pueden corresponder a una mascota o a un veterinario. JpaRepository aporta operaciones de persistencia sin escribirlas desde cero.

Instrucciones paso a paso

1. Crea en MySQL Workbench una base vacía llamada vetturmo y configura la conexión local desde application.properties.
2. Define Propietario, Mascota, Veterinario y Cita con id generado, campos del contrato, constructor vacío y accesores necesarios para JPA.

- 3. Modela que muchas mascotas pertenecen a un propietario. Decide si necesitas navegar también desde el propietario hacia sus mascotas y justifica la decisión.
- 4. Modela que muchas citas pertenecen a una mascota y a un veterinario. Ubica las llaves foráneas en la tabla que representa la cita.
- 5. Crea un repositorio por entidad con la entidad y el tipo de id correctos.
- 6. Arranca la aplicación, inspecciona el SQL y confirma en Workbench las tablas, llaves primarias y llaves foráneas.
- 7. Evita cargar datos definitivos directamente en Workbench: el flujo se probará desde la API.

Responsabilidad de los archivos involucrados

Archivo	Responsabilidad
model/Propietario.java	Representa a la persona responsable de una o varias mascotas.
model/Mascota.java	Representa al paciente y conserva la referencia a su propietario.
model/Veterinario.java	Representa al profesional que puede recibir citas.
model/Cita.java	Une fecha, motivo, mascota y veterinario en una reserva persistente.
repository/...Repository.java	Proporciona operaciones de acceso a datos para cada entidad.
application.properties	Define la conexión MySQL y el comportamiento de Hibernate durante el desarrollo.

Resultado esperado

Punto de control 2

Hibernate crea o actualiza el esquema. Las llaves foráneas aparecen en mascotas y citas, el modelo no contiene relaciones innecesarias y los repositorios compilan.

Lista breve de verificación

- ☐ Cada entidad tiene id generado y constructor vacío.
- ☐ Las cardinalidades corresponden a la historia de la veterinaria.
- ☐ Las llaves foráneas se encuentran en el lado “muchos”.
- ☐ Los repositorios usan tipos correctos.
- ☐ MySQL muestra las tablas y relaciones esperadas.

Evidencia que debes presentar

- Diagrama sencillo del modelo o captura del esquema relacional.
- Captura del SQL generado al iniciar y de las tablas en Workbench.
- Explicación de la diferencia entre JPA, Hibernate y JpaRepository.

Parte 3 — Construir el flujo REST por capas

Propósito

Exponer responsables, mascotas y veterinarios mediante endpoints coherentes, conectando controller, service y repository sin entregar entidades completas al cliente.

Conceptos que se repasan

El controller traduce la petición HTTP; el service organiza las decisiones del negocio; el repository accede a MySQL. La inyección por constructor permite que Spring entregue las dependencias. Los DTO separan el contrato público de la estructura persistente y ayudan a evitar recursión JSON.

Instrucciones paso a paso

1. Implementa los servicios para crear y listar propietarios, mascotas y veterinarios. Mantén en el service la resolución de relaciones recibidas como id.
2. Crea los DTO de entrada y salida definidos en el contrato. La respuesta de Mascota debe identificar al propietario sin devolver toda su colección.
3. Crea los controllers con rutas base en plural y conecta cada acción con el service mediante inyección por constructor.
4. Implementa los endpoints GET y POST de esta parte y usa ResponseEntity para expresar 200 y 201 con intención.
5. Prueba el orden natural: crear propietario, crear mascota relacionada, crear veterinario y consultar las listas.
6. Comprueba que ninguna respuesta muestre contraseñas, colecciones innecesarias o ciclos entre objetos.
7. Reinicia la aplicación y repite las consultas para comprobar persistencia.

Responsabilidad de los archivos involucrados

Archivo o grupo	Responsabilidad
service/PropietarioService.java	Crea y consulta responsables.
service/MascotaService.java	Valida la referencia al propietario y administra mascotas.
service/VeterinarioService.java	Crea y consulta profesionales de la clínica.
controller/...Controller.java	Expone el contrato HTTP y delega la lógica.
dto/...Request.java	Define únicamente los datos aceptados en una creación.
dto/...DTO.java	Entrega respuestas planas, legibles y sin relaciones recursivas.

Resultado esperado

Punto de control 3

Una secuencia de peticiones crea un propietario, su mascota y un veterinario. Las respuestas usan DTO, los datos sobreviven al reinicio y ninguna relación produce recursión JSON.

Lista breve de verificación

- ☐ Los controllers no acceden directamente a los repositorios.
- ☐ Los services no contienen detalles propios de HTTP.
- ☐ Las relaciones se resuelven con registros existentes.
- ☐ La salida usa DTO y no entidades completas.
- ☐ La persistencia resiste un reinicio.

Evidencia que debes presentar

- Peticiones de creación y consulta de las tres entidades.
- Respuesta de una mascota con información breve de su propietario.
- Demostración de los datos antes y después de reiniciar el servidor.

Parte 4 — Agendar citas sin cruces

Propósito

Completar el flujo principal de VetTurno: crear una cita válida, evitar horarios duplicados y consultar la agenda por veterinario.

Conceptos que se repasan

Una regla de negocio expresa una condición del mundo real que no depende del formato HTTP. Por eso la comprobación de horario vive en el service. Una consulta derivada permite preguntar al repositorio si ya existe una cita para un veterinario en una fecha y hora determinadas.

Instrucciones paso a paso

1. Crea CitaRequest y CitaDTO según el contrato. La entrada recibe ids; la salida muestra datos comprensibles de la mascota, su responsable y el veterinario.
2. Añade al repositorio de citas una consulta derivada para detectar la coincidencia de veterinario y fechaHora. Añade otra para listar citas por veterinario.
3. En el service, busca la mascota y el veterinario antes de guardar. Rechaza la operación si alguna referencia no existe.
4. Comprueba que la fecha sea futura y que el veterinario no tenga otra cita exactamente en ese horario.
5. Crea los endpoints para agendar, listar toda la agenda y filtrar por veterinario.
6. Prueba una cita válida, una fecha pasada y un segundo intento con el mismo veterinario y horario.
7. Reinicia la aplicación y confirma que la cita válida continúa almacenada.

Responsabilidad de los archivos involucrados

Archivo	Responsabilidad
dto/CitaRequest.java	Recibe fecha, motivo y referencias mínimas para agendar.
dto/CitaDTO.java	Presenta la cita de manera plana y útil para recepción.
repository/CitaRepository.java	Consulta agenda, coincidencias de horario y citas por veterinario.
service/CitaService.java	Resuelve relaciones y aplica las reglas de fecha y disponibilidad.
controller/CitaController.java	Expone creación y consultas sin contener la decisión del negocio.

Resultado esperado

Punto de control 4

VetTurno agenda una cita futura, rechaza el cruce de horario y devuelve una agenda por veterinario con datos planos y comprensibles.

Lista breve de verificación

- ☐ La cita relaciona registros reales de mascota y veterinario.
- ☐ La fecha pasada se rechaza antes de persistir.
- ☐ Un mismo veterinario no puede recibir dos citas a la misma hora.
- ☐ La consulta filtrada devuelve solo las citas correspondientes.
- ☐ CitaDTO no expone entidades completas.

Evidencia que debes presentar

- Creación exitosa de una cita con estado 201.

- Respuesta controlada ante fecha pasada y horario duplicado.
- Consulta de agenda filtrada por veterinario.

Parte 5 — Proteger la información con JWT y roles

Propósito

Identificar a quien usa VetTurno, proteger las contraseñas y limitar el registro de veterinarios al rol ADMIN, sin crear sesiones en el servidor.

Conceptos que se repasan

Autenticar significa comprobar identidad; autorizar significa decidir qué puede hacer esa identidad. BCrypt almacena una huella irreversible de la contraseña. El JWT es una credencial firmada —no cifrada— enviada como Bearer token. En una API stateless, cada petición protegida presenta su propia credencial.

Instrucciones paso a paso

1. Crea Rol con USER y ADMIN, y Usuario con email único, contraseña y rol almacenado como texto.
2. Crea el repositorio de usuarios con búsqueda por email y verificación de existencia.
3. Define RegistroRequest, LoginRequest y AuthResponse. Nunca devuelvas la entidad Usuario.
4. Implementa el registro para asignar USER y aplicar BCrypt. Implementa el login usando el mecanismo de autenticación de Spring y entrega un JWT vigente.
5. Configura el servicio JWT, la carga del usuario y el filtro que revisa Authorization: Bearer antes de cada petición.
6. Deja públicos únicamente registro y login. Permite el trabajo de recepción a USER y ADMIN, y restringe POST /api/veterinarios a ADMIN.
7. Registra a Paula, comprueba el hash, intenta crear un veterinario y confirma el 403. Habilita de forma controlada a Doña Marta como ADMIN, inicia sesión de nuevo y repite con éxito.

Responsabilidad de los archivos involucrados

Archivo	Responsabilidad
model/Usuario.java y Rol.java	Identidad persistente y conjunto cerrado de permisos.
repository/UsuarioRepository.java	Busca la identidad por email.
dto/RegistroRequest.java, LoginRequest.java y AuthResponse.java	Contratos mínimos de autenticación.
service/AuthService.java	Orquesta registro, hashing, autenticación y emisión del token.
security/JwtService.java	Genera, firma, interpreta y valida JWT.
security/UsuarioDetailsService.java	Adapta Usuario al formato que comprende Spring Security.
security/JwtAuthFilter.java	Lee el Bearer token y establece la identidad de la petición.
security/SecurityConfig.java	Define rutas, roles, modo stateless y componentes de seguridad.
controller/AuthController.java	Expone registro y login.

Resultado esperado

Punto de control 5

El login entrega un JWT utilizable, la contraseña es ilegible en MySQL, USER recibe 403 al registrar veterinarios y ADMIN completa la misma operación con estado 201.

Lista breve de verificación

- ☐ La contraseña nunca se guarda ni responde en texto plano.
- ☐ El registro asigna USER sin aceptar el rol del cliente.
- ☐ El JWT tiene expiración y se envía con Bearer.
- ☐ La política de sesiones es stateless.
- ☐ El permiso de ADMIN se demuestra con un 403 y un 201.

Evidencia que debes presentar

- Hash visible en MySQL, con la contraseña original oculta.
- Respuesta de login con el token parcialmente oculto.
- Comparación: POST /api/veterinarios con USER → 403; con ADMIN → 201.

Parte 6 — Validar y responder errores con claridad

Propósito

Evitar que información incoherente llegue a MySQL y ofrecer respuestas de error consistentes, comprensibles y seguras.

Conceptos que se repasan

Bean Validation declara reglas en los DTO de entrada y @Valid activa su revisión. Un manejador global centraliza excepciones para que todos los endpoints hablen el mismo lenguaje. ResponseEntity comunica el resultado mediante un cuerpo y un estado HTTP adecuados.

Instrucciones paso a paso

1. Define validaciones en los DTO de propietario, mascota, veterinario y cita según el contrato funcional.
2. Usa una regla de fecha futura en CitaRequest y valida que los ids requeridos estén presentes.
3. Añade al registro un email obligatorio y válido, y una contraseña obligatoria con longitud mínima.
4. Activa la validación en los puntos de entrada con la anotación correspondiente.
5. Crea ApiError con status, mensaje, errores por campo y timestamp.
6. Crea un manejador global para transformar validaciones y reglas del negocio en 400, y fallos imprevistos en 500 con un mensaje genérico.
7. Prueba campos vacíos, email inválido, fecha pasada, relaciones inexistentes y horario duplicado. Comprueba que ninguna respuesta filtre clases, rutas internas o trazas.

Responsabilidad de los archivos involucrados

Archivo	Responsabilidad
dto/...Request.java	Declara las reglas de calidad de cada entrada.
exception/ApiError.java	Mantiene un formato estable para los errores visibles.
exception/GlobalExceptionHandler.java	Traduce excepciones de toda la API a respuestas controladas.
service/CitaService.java	Genera errores de negocio cuando la fecha o el horario no son válidos.
Controllers	Activan la validación y seleccionan estados HTTP sin duplicar el manejo de errores.

Resultado esperado

Punto de control 6

MySQL no recibe registros inválidos. Las respuestas 400 explican qué debes corregir, las creaciones válidas responden 201 y un error inesperado no expone detalles internos.

Lista breve de verificación

- ☐ Las reglas de formato viven en DTO de entrada.
- ☐ Las reglas de agenda viven en el service.
- ☐ ApiError conserva el mismo formato entre casos.
- ☐ Los estados 200, 201, 400, 403 y 500 se usan con intención.
- ☐ No se muestran trazas ni nombres internos al cliente.

Evidencia que debes presentar

- Una respuesta 400 con varios errores de campos.
- Una respuesta 400 por cruce de horario.
- Explicación de por qué el manejador global no reemplaza las reglas de seguridad.

Parte 7 — Documentar, probar y publicar

Propósito

Convertir VetTurno en una entrega comprensible y verificable mediante Swagger/OpenAPI, pruebas manuales, documentación accesible y un repositorio GitHub reproducible.

Conceptos que se repasan

OpenAPI describe el contrato y Swagger UI permite recorrerlo y ejecutarlo. Las pruebas manuales cubren éxitos, validaciones y permisos. Git conserva la evolución del proyecto y GitHub publica la entrega. Como el MVP no tiene interfaz propia, la accesibilidad se aplica a títulos, tablas, mensajes, descripciones y evidencias.

Instrucciones paso a paso

1. Configura en OpenAPI el nombre VetTurno, la descripción de Veterinaria Huellitas y el esquema Bearer JWT.
2. Permite las rutas técnicas de Swagger en Spring Security sin abrir los endpoints de negocio.
3. Ejecuta la matriz de pruebas y registra resultado real, estado HTTP y evidencia. Si un caso falla, corrige la aplicación antes de marcarlo como aprobado.
4. Completa el README con historia, alcance, tecnologías, modelo, endpoints, roles, configuración de MySQL, orden del flujo, pruebas y errores frecuentes.
5. Revisa la accesibilidad: encabezados jerárquicos, tablas con títulos, mensajes que no dependan del color y capturas acompañadas por una descripción.
6. Empaqueta el proyecto con Maven, revisa que Git no contenga secretos ni target y realiza commits con mensajes comprensibles.
7. Crea el repositorio remoto, conecta origin, publica la rama principal y verifica el enlace desde una sesión donde no dependas de tu caché local.

```
.\mvnw.cmd clean package -DskipTests    # Windows
```

```
git remote add origin URL_REPOSITORIO · git branch -M main · git push -u origin main
```

Responsabilidad de los archivos involucrados

Archivo o recurso	Responsabilidad
config/OpenApiConfig.java	Describe la identidad de la API y su esquema Bearer.
security/SecurityConfig.java	Permite Swagger sin debilitar las rutas protegidas.
README.md	Explica instalación, configuración, contrato, roles, pruebas y evidencias.
docs/evidencias/ (sugerido)	Organiza capturas o archivos con nombres y descripciones claras.
.gitignore	Evita publicar archivos generados, locales o sensibles.
Repositorio GitHub	Es el punto oficial de publicación del taller.

Resultado esperado

Punto de control 7

Una persona que no conoce el código puede clonar VetTurno, configurarlo, iniciar sesión, probar el flujo desde Swagger y comprender los resultados mediante el README.

Lista breve de verificación

- ☐ Swagger abre y muestra todos los endpoints obligatorios.
- ☐ Authorize agrega el token a las peticiones protegidas.
- ☐ La matriz contiene resultados reales y evidencia.
- ☐ El README no revela contraseñas, secretos ni tokens.
- ☐ El repositorio puede clonarse y seguirse desde cero.

Evidencia que debes presentar

- Captura de Swagger con la información de VetTurno y el botón Authorize.
- Matriz completa dentro del README o en un archivo enlazado.
- Enlace al repositorio GitHub y captura de una ejecución siguiendo el README.

Matriz mínima de pruebas manuales

#	Escenario	Resultado esperado
1	La aplicación inicia con MySQL disponible.	Servidor activo y esquema accesible.
2	Registro válido de Paula.	200 y token; contraseña hasheada.
3	Registro con email inválido y clave corta.	400 con errores por campo.
4	Login con credenciales válidas.	200 y JWT vigente.
5	GET /api/citas sin token.	Acceso rechazado.
6	POST /api/veterinarios con USER.	403 Forbidden.
7	POST /api/veterinarios con ADMIN.	201 y veterinario persistido.
8	Creación válida de propietario.	201 y DTO sin colecciones anidadas.
9	Creación de mascota con propietario existente.	201 y relación correcta.
10	Mascota con propietario inexistente.	400 controlado; no se inserta fila.
11	Cita futura con referencias válidas.	201 y cita persistida.
12	Cita con fecha pasada.	400 con mensaje claro.

#	Escenario	Resultado esperado
13	Segundo intento con mismo veterinario y horario.	400; se conserva una sola cita.
14	Filtro de citas por veterinario.	200 y solo coincidencias.
15	Reinicio y prueba desde Swagger con Authorize.	Datos persisten y flujo protegido funciona.

Prompts de IA permitidos

Puedes adaptar estos prompts para comprender, revisar o diagnosticar. No los uses para solicitar la solución completa. Conserva en el README una nota breve sobre qué consultaste y cómo comprobaste las sugerencias.

Momento	Prompt sugerido
Comprender capas	“Explicame controller, service y repository usando VetTurno. No generes código; dame una analogía y señales de responsabilidades mezcladas.”
Revisar relaciones	“Revisa esta descripción de Propietario, Mascota, Veterinario y Cita. Señala problemas de cardinalidad sin escribir anotaciones ni clases.”
Diagnosticar JSON	“Formula hipótesis sobre esta recursión JSON y dime qué relaciones o DTO debo inspeccionar; no escribas la corrección.”
Revisar la agenda	“Contrasta mis reglas de fecha y cruce de horario con este comportamiento. Propón comprobaciones, no una implementación.”
Auditar seguridad	“Compara mis rutas y roles con esta regla: recepción para USER/ADMIN y alta de veterinarios solo ADMIN. No generes SecurityConfig.”
Revisar validaciones	“Señala reglas ausentes o redundantes en estos DTO, sin producir código Java.”

Límite de uso

No entregues código que no puedas explicar. La IA puede equivocarse en versiones, relaciones y permisos. Tu responsabilidad es contrastar sus sugerencias con el comportamiento real de VetTurno y con el material de clase.

Rúbrica de evaluación — 100 puntos

La calificación prioriza el funcionamiento y la comprensión de los temas impartidos. Docker y cualquier mejora opcional tienen cero puntos y no compensan fallas en el alcance obligatorio.

Criterio	Desempeño esperado para el puntaje completo	Puntos
1. Proyecto y arquitectura	Maven y Java 17 correctos; app inicia; paquetes por responsabilidad; inyección por constructor; nombres coherentes con VetTurno.	10
2. JPA, Hibernate y MySQL	Cinco entidades; ids generados; relaciones justificadas; llaves foráneas; repositorios; esquema y persistencia correctos.	18
3. API REST y DTO	Flujo controller-service-repository; endpoints obligatorios; GET/POST; DTO de entrada/salida; JSON plano; estados 200/201.	20
4. Flujo de citas	Relaciones resueltas; fecha futura; prevención del horario duplicado; consulta derivada y filtro por veterinario.	12
5. Seguridad JWT y roles	Registro USER, BCrypt, login, JWT, filtro, usuario cargado, stateless y alta de veterinarios solo ADMIN; prueba 403.	18
6. Validaciones y errores	Reglas completas; @Valid activo; ApiError consistente; manejo global de 400 y 500; sin detalles internos.	12

Criterio	Desempeño esperado para el puntaje completo	Puntos
7. Swagger, pruebas y publicación	OpenAPI y Authorize; 15 pruebas con evidencia; README reproducible; GitHub sin secretos; documentación accesible.	10

Nivel	Referencia para asignar puntaje dentro de cada criterio
Completo · 100% del criterio	Funciona, corresponde al alcance, está documentado y el estudiante lo explica con evidencia.
Parcial sólido · 70–85%	Funciona lo principal, pero existen omisiones menores, evidencia incompleta o explicación imprecisa.
Básico · 40–60%	Hay piezas aisladas o funcionamiento inestable; la integración y la comprensión son limitadas.
Insuficiente · 0–30%	No funciona, falta la mayor parte, se desvía del alcance o no puede defenderse.

Comprobación matemática

$10 + 18 + 15 + 12 + 18 + 12 + 10 + 5 = 100$ puntos.

Lista final de entrega

- ☐ Repositorio GitHub accesible con la rama principal actualizada.
- ☐ README con historia, alcance, tecnologías, arquitectura, configuración, endpoints, roles, pruebas y evidencias.
- ☐ Proyecto Maven con Java 17 y las dependencias trabajadas en el módulo.
- ☐ Entidades Propietario, Mascota, Veterinario, Cita y Usuario con relaciones y repositorios.
- ☐ Servicios, controllers y DTO sin exponer entidades completas.
- ☐ Regla de fecha futura y prevención de cita duplicada por veterinario y horario.
- ☐ Registro, login, BCrypt, JWT, filtro, stateless y roles USER/ADMIN.
- ☐ Validaciones, @Valid, ApiError, manejador global y estados HTTP correctos.
- ☐ Swagger/OpenAPI con Authorize y rutas realmente protegidas.
- ☐ Matriz de 15 pruebas manuales con resultados reales.
- ☐ Configuración local documentada sin contraseñas ni secretos publicados.

Secuencia recomendada para entregar

1. Haz una ejecución desde cero siguiendo tu propio README.
2. Ejecuta las pruebas críticas y confirma que la evidencia corresponde a la versión publicada.
3. Revisa el repositorio como si fueras la persona evaluadora: enlaces, instrucciones y ausencia de secretos.
4. Copia en la portada las URLs finales y prepara la demostración.
5. Entrega cuando el commit mostrado en la demostración sea el mismo publicado en GitHub.

Preguntas para pensar

#	Pregunta
1	¿Qué trabajo repetitivo evita Spring Boot y qué responsabilidad sigue siendo tuya?
2	¿Qué diferencia práctica hay entre pom.xml y application.properties?
3	Segue POST /api/citas: ¿qué capa recibe la petición y cuál persiste?
4	¿Por qué inyectas dependencias por constructor en lugar de crear objetos con new?
5	¿Qué diferencia existe entre JPA, Hibernate y JpaRepository?
6	¿Dónde quedan las llaves foráneas del modelo y por qué?
7	¿Qué evita MascotaDTO frente a devolver Mascota directamente?
8	¿Cómo se produciría una recursión JSON entre propietario y mascotas?
9	¿Por qué la validación de fecha y la disponibilidad viven en CitaService?
10	¿Cómo detectas un horario ocupado mediante el repositorio?
11	Explica autenticación y autorización usando el 403 de Paula.
12	¿Por qué un JWT firmado no debe contener secretos?
13	¿Qué aporta BCrypt y por qué la contraseña no se descifra?
14	¿Qué significa stateless y qué acompaña cada petición protegida?
15	¿Por qué @Valid es necesario si los campos ya tienen reglas?
16	¿Qué ventaja tiene un manejador global frente a responder errores en cada controller?
17	¿Por qué agendar una cita responde 201 y consultarla responde 200?
18	¿Qué diferencia existe entre OpenAPI y Swagger UI?
19	¿Cómo demuestras que la cita persiste después de reiniciar?
20	¿Qué mejora futura tendría valor para Doña Marta y por qué quedó fuera del MVP?

Cierre

No necesitas construir la veterinaria más grande del mundo. Necesitas demostrar que sabes escuchar un problema cercano, convertirlo en una API organizada y explicar por qué cada pieza existe.

Actividad extra sin puntaje — Ejecutar VetTurno con Docker

Importante

Esta actividad es voluntaria, vale 0 puntos y no forma parte de la entrega obligatoria ni de la defensa. Realízala únicamente cuando el taller principal esté terminado y estable.

Para qué sirve

Docker permite empaquetar la aplicación y su entorno de ejecución en una imagen reproducible. El Dockerfile describe cómo construirla; la imagen es el resultado y el contenedor es una ejecución concreta de esa imagen.

Paso a paso

1. Instala y abre Docker Desktop. Comprueba desde la terminal que el comando docker responde.
2. Genera el JAR de VetTurno y confirma que aparece dentro de target.
3. Crea un archivo Dockerfile en la raíz. Debe partir de una imagen de ejecución con Java 17, copiar el JAR, declarar el puerto y arrancar la aplicación. No copies credenciales dentro del archivo.
4. Crea .dockerignore para excluir target innecesario, archivos del IDE, Git y configuración sensible del contexto de construcción.
5. Construye la imagen desde la raíz y revisa que el proceso termine sin errores.
6. Ejecuta el contenedor publicando el puerto 8080 e inyectando la conexión MySQL y el secreto JWT mediante variables.
7. Si MySQL está instalado en tu equipo, usa host.docker.internal en lugar de localhost desde el contenedor.
8. Abre Swagger, inicia sesión y prueba una consulta. Guarda una captura solo como evidencia personal del reto adicional.

```
.\mvnw.cmd clean package -DskipTests
```

```
docker build -t veturno-api .
```

```
docker run -p 8080:8080 -e  
DB_URL="jdbc:mysql://host.docker.internal:3306/veturno" -e DB_USER=TU_USUARIO -e  
DB_PASSWORD=TU_PASSWORD -e JWT_SECRET=TU_SECRETO veturno-api
```

Comprobación voluntaria

- ☐ La imagen se construye sin copiar secretos.
- ☐ El contenedor inicia VetTurno en el puerto esperado.
- ☐ La aplicación alcanza MySQL desde el contenedor.
- ☐ Swagger y al menos un endpoint responden correctamente.
- ☐ Comprendes la diferencia entre Dockerfile, imagen y contenedor.